C++ Programming

Juha Yli-Hemminki

20.11.2025

SeAMK

SeAMK

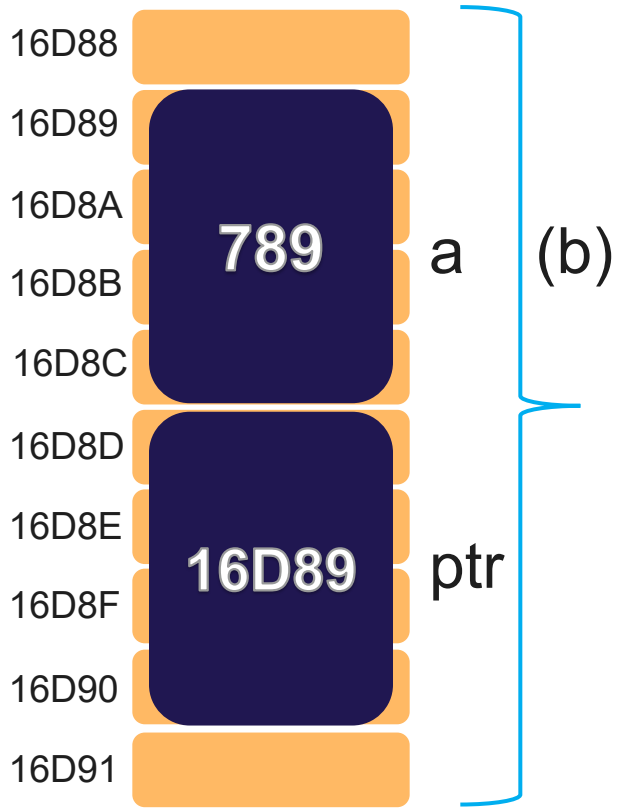
Computer memory

Memory address

```
int a = 789;

Reference:
int& b = a; // alias for a

Pointer:
int* ptr = &a; // gets the value 16D89
```



Computer memory

SeAMK

Computer memory

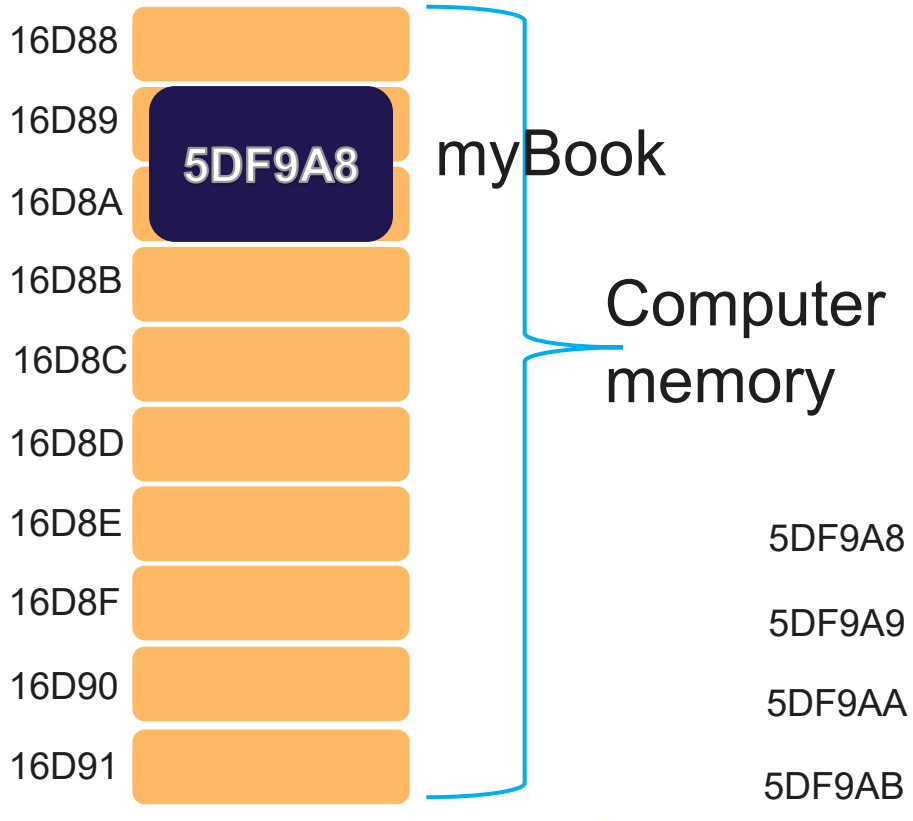
Memory address

Pointer:

Book* myBook;

Creating new object returns the memory address:

myBook = new Book();



SeAMK

Class vs Object

- **Objects** are created from **classes** during program execution
- The operation of the program is built on the cooperation of objects
- Objects may receive action requests and request actions from other objects
- Object may change its member values by the methods called from outside

SeAMK

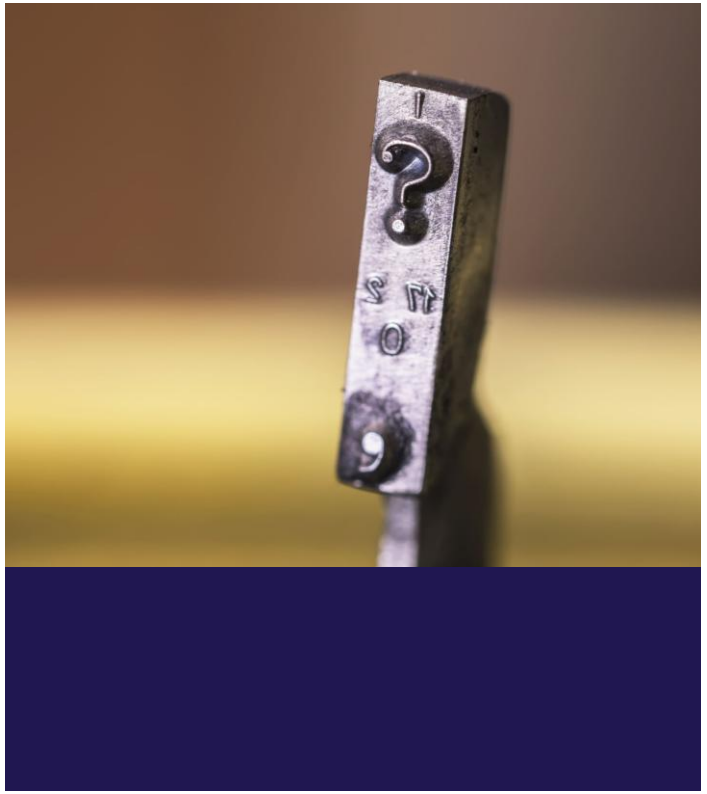
Class vs Object

- It may be considered that a class serves as a model by which objects are made
- Sometimes a class is compared to a stamp and an object is a stamp made with a stamp, respectively
- Class is written into the code and object is created when program is run

SeAMK

Class vs Object

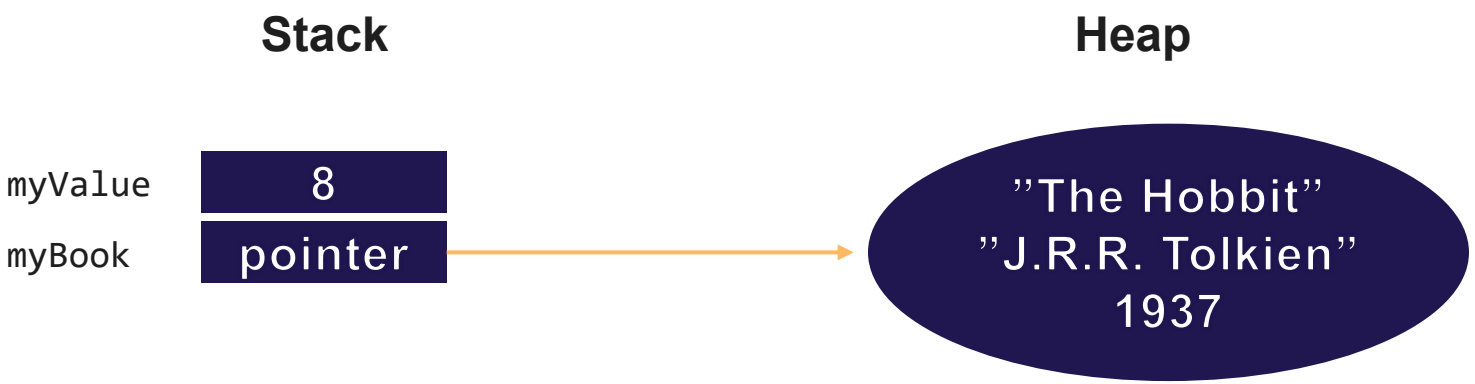
- Variables and functions are defined in the class, which become properties of the object created from it
- A class written in program code is used only through objects made from it
- If no object is created from the class written in the code, it is not used in the program at all



SeAMK

Stack and Heap

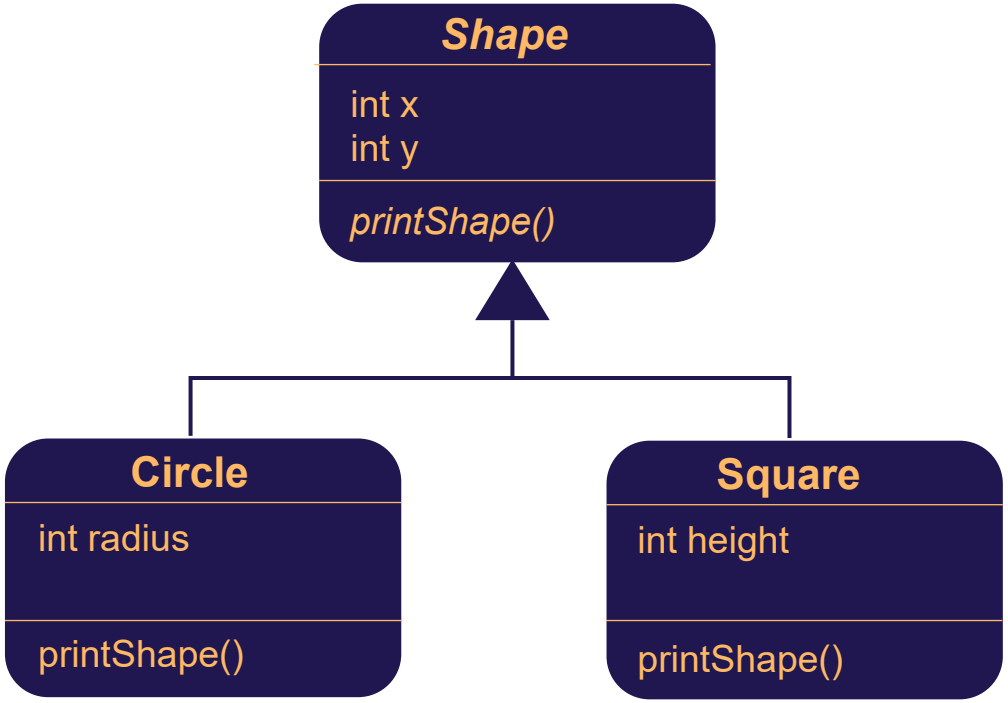
```
int myValue = 8;
Book* myBook = new Book(”The Hobbit", "J.R.R. Tolkien”, 1937);
```



SeAMK

Inheritance

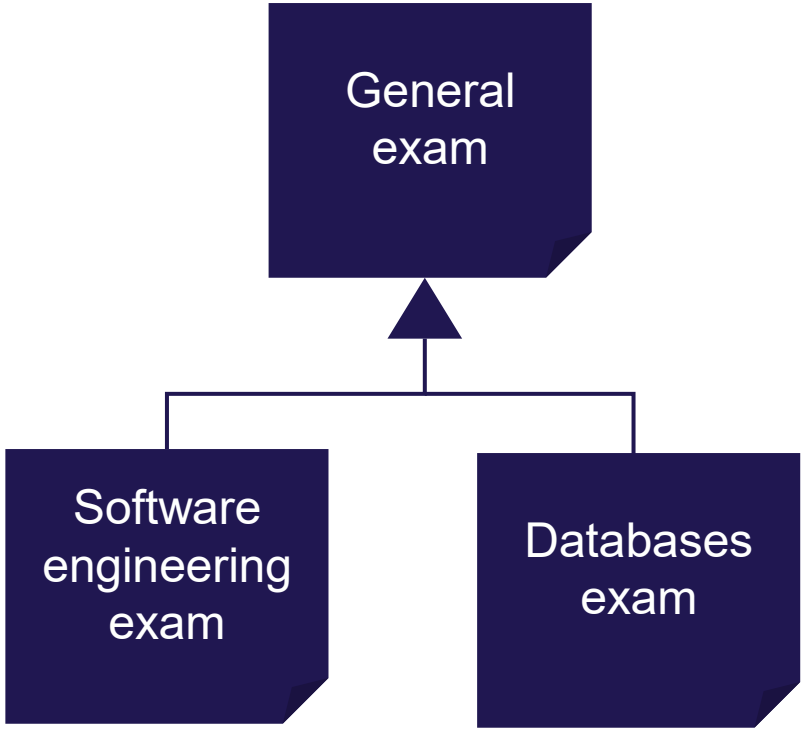
- Parent class: Shape
- Child classes:
  Circle and Square



SeAMK

Inheritance

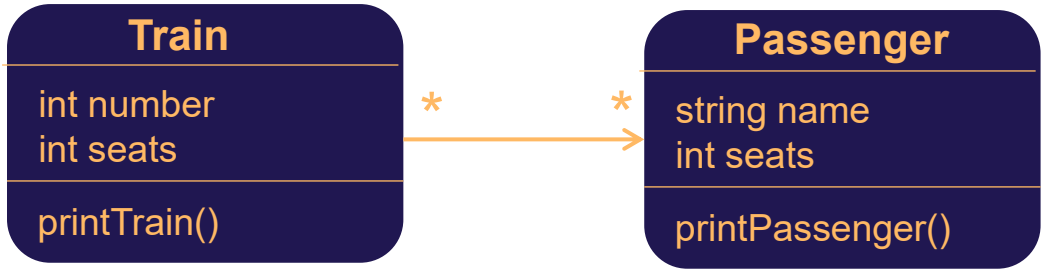
- A bit same as having some base document with general data
- Then making sub documents according the base document and adding some new data



SeAMK

Association

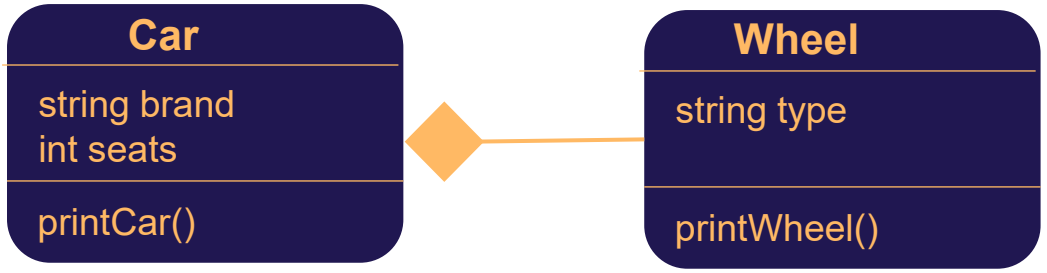
- Passenger can be called from Train
- Train has reference (a pointer) to Passenger object



SeAMK

Composition

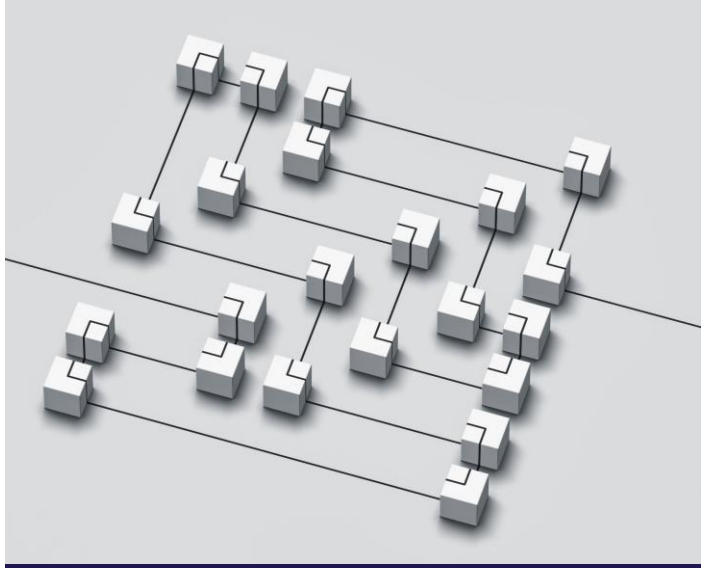
- Wheel does not exist without a Car
- Wheels are created for the Car



SeAMK

UML (Unified Modelling Language)

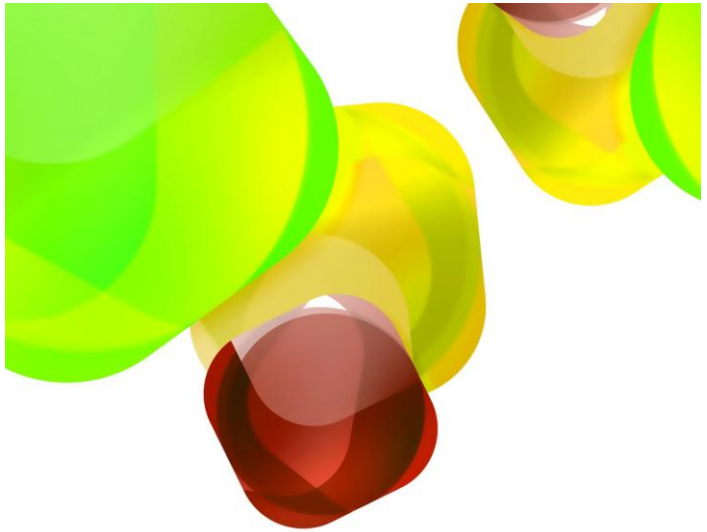
- Graphs for planning structure for the program
- Use case diagrams to determine requirements and different actors
- Class diagram to plan class structure
- Activity diagrams to describe what steps different operations need for completing



SeAMK

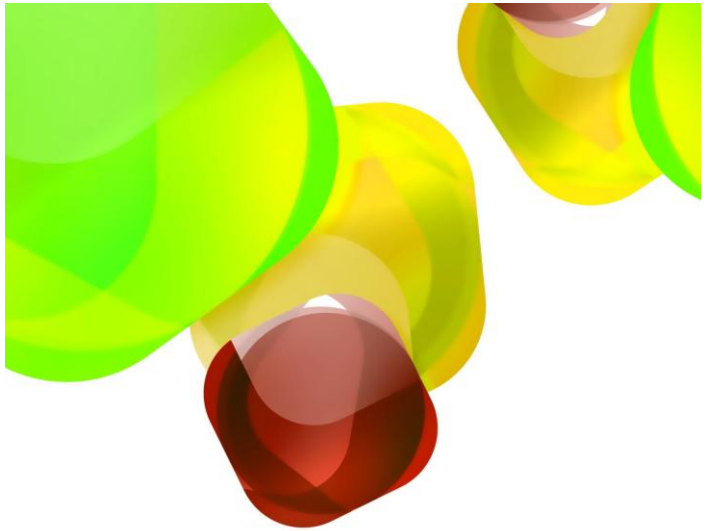
Abstract class

- The class including at least one abstract method is defined as abstract
- Abstract class can not be used for creating new object
  - Abstract class can be used only by inheritance
  - Abstract class combines common features, makes them visible and possible for other classes to inherit
  - Can be used as base type for child class objects



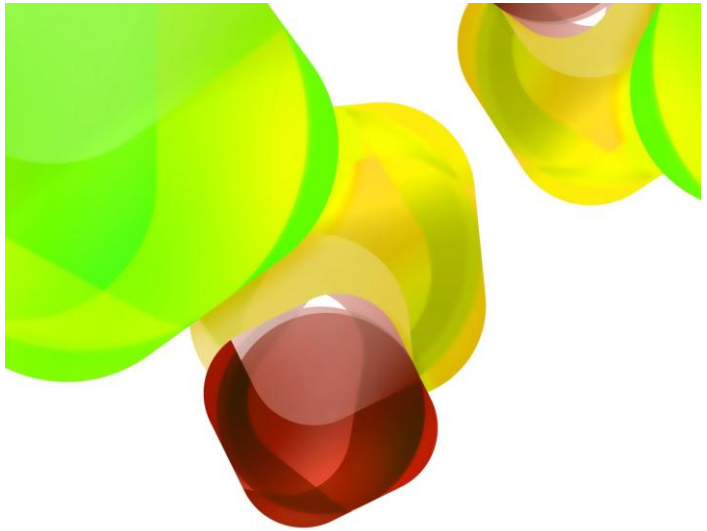
Abstract class

- Abstract method does not have implementation
  - Implementation needs to be added in derived class (child class)



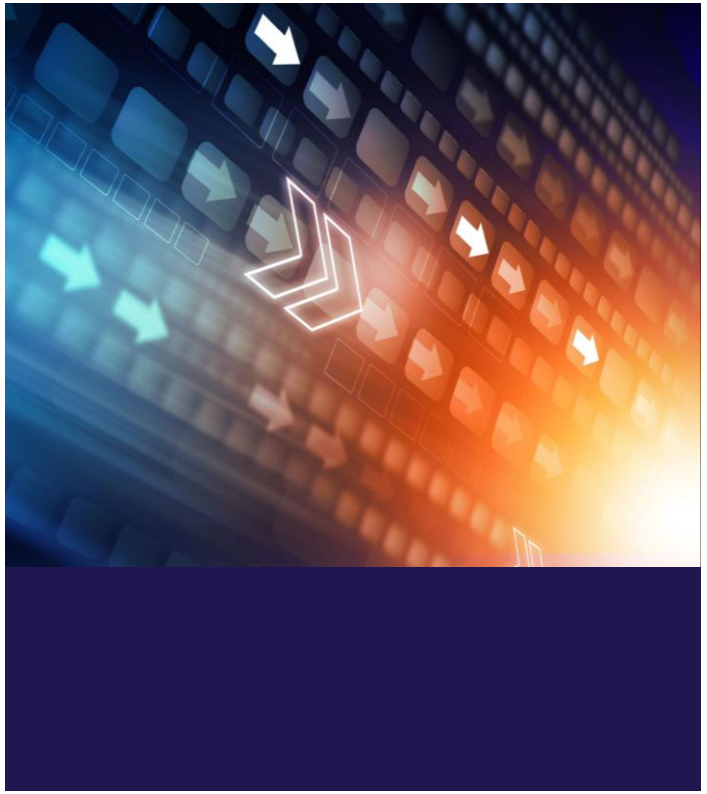
Abstract class

- Pointer of parent class type may store address to any child class type object
- Abstract class destructor needs to be declared **virtual**
- Otherwise deleting object using parent class pointer will call parent class destructor



Function overload and override

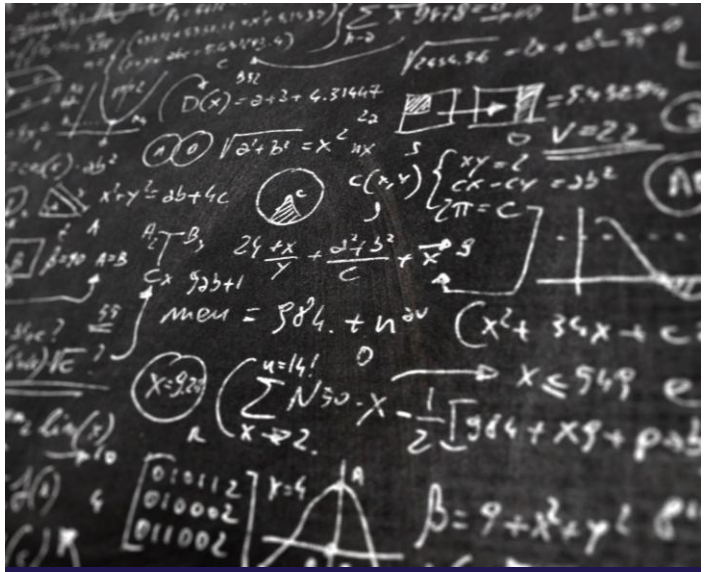
- Function overload means multiple functions with same name but different input parameters/return value
- Function override means new implementation for the function in inheriting class



SeAMK

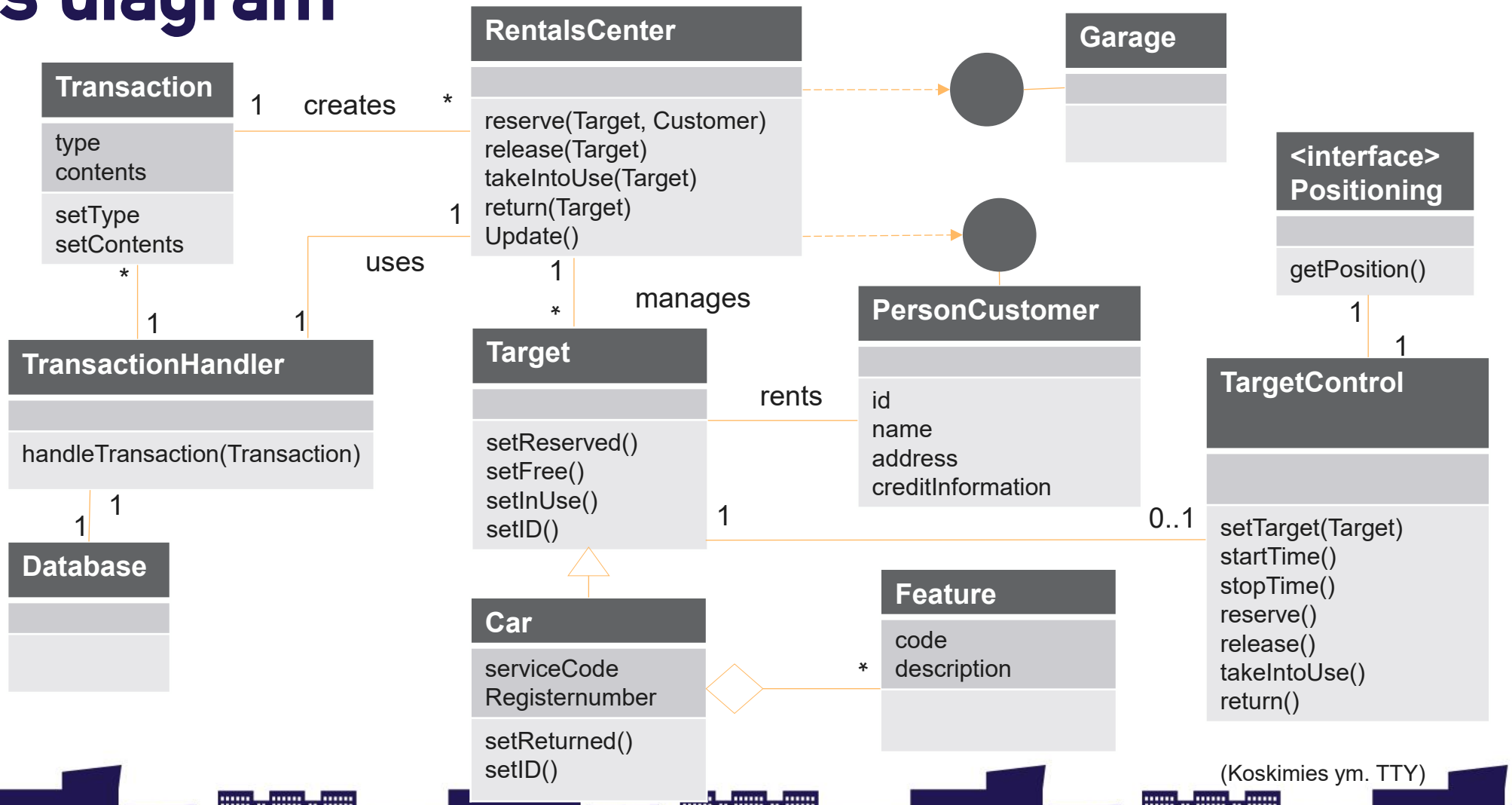
Class diagram

- Describes the static structure of a system at the class level
- The class diagram can also be used to see what kind of object structures the system can have at runtime
- For this reason, the numbers are also shown in the relations between classes



SeAMK

Class diagram



(Koskimies ym. TTY)

SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES